# Evaluating Meta-heuristic Efficiency in the Context of the Generalized Assignment Problem

Technical Report submitted for partial fulfillment of requirement of Course
**Evolutionary Computation**


## Department of Computer Science and Engineering
## National Institute of Technology, Jamshedpur

**Submitted by:-**

**Piyush Sahu (2025PGCSCS16)**

**Yashraj Gupta(2025PGCSCS06)**

**Submitted to:-**

**Dr Dinesh Kumar**

**Assistant Professor**

**Computer Science and Engineering**

**Abstract** **3**
**1. Introduction** **4**
**2. Background** **4**
2.1 Generalized Assignment Problem (GAP) 4
Constraints Explanation 5
Applications 5
2.2 Metaheuristics Overview 5
**3. Problem Definition** **6**
**4. Comparative Study of Metaheuristic Techniques** **7**
4.1 Binary Genetic Algorithm (BGA): 7
4.2 Real-Coded Genetic Algorithm (RCGA): 7
4.3 Particle Swarm Optimization (PSO): 7
4.4 Teaching-Learning-Based Optimization (TLBO): 7
4.5 Differential Evolution (DE): 7
4.6 Artificial Bee Colony (ABC): 7

**5. Experimental Analysis** **8**
5.1 Parameter Settings 8
5.1.1 Common Parameters 8
5.1.2 Algorithm-Specific Parameters 8
5.2 Constraint Handling and Variation Strategy 11
Constraint Handling 11
Variation Strategy 12
5.3 Results and Discussion 12
5.3.1 Table of Comparison 12
5.3.2 Convergence Curves 12
5.3.3 Final fitness comparison 15
5.3.4 Execution time comparison across instances 15
5.4 Discussion 16
**6. Conclusion** **17**

# Abstract

This study investigates the effectiveness of various metaheuristic strategies in solving the Generalized Assignment Problem (GAP), a classic NP-hard challenge in resource allocation. We implemented and benchmarked seven distinct algorithms—ranging from population-based methods like **PSO** and **ABC** to evolutionary approaches such as **BGA**, **RCGA**, and **Differential Evolution**. By testing these against the **OR-Library GAP12** instance, we analyzed how each method balances exploration and exploitation. Our findings suggest that while standard implementations vary in efficiency, the integration of problem-specific heuristics is crucial for reaching near-optimal result

# 1. Introduction

Combinatorial optimization deals with selecting the best possible solution from a limited yet often extremely large set of alternatives. Such problems are commonly encountered in practical areas like scheduling, logistics, resource management, and network design. As the size of the problem increases, the number of possible solutions grows rapidly, making it difficult to apply exact algorithms efficiently for large-scale instances.

A well-known example in this domain is the Generalized Assignment Problem (GAP). It involves allocating a set of tasks to multiple agents in a way that minimizes overall cost while respecting the capacity limits of each agent. This problem has applications in fields such as job scheduling, transportation planning, and resource allocation. GAP is categorized as NP-hard, indicating that finding an exact optimal solution becomes computationally infeasible as the problem size grows.

To handle such complexity, researchers often rely on metaheuristic methods. These approaches are designed to navigate large search spaces effectively and produce high-quality approximate solutions within a reasonable time frame. Popular techniques include Genetic Algorithms, Simulated Annealing, Tabu Search, and Particle Swarm Optimization, all of which have been widely applied to NP-hard problems like GAP.

This study focuses on implementing and evaluating several metaheuristic strategies for solving the Generalized Assignment Problem. The methods are compared in terms of solution accuracy, convergence speed, and computational performance to determine the most suitable approach for tackling this type of optimization challenge.

# 2. Background

### 2.1 Generalized Assignment Problem (GAP)

The Generalized Assignment Problem, frequently abbreviated as GAP, represents a fundamental challenge within the realm of discrete optimization. At its core, the problem involves the strategic allocation of a designated collection of tasks across a group of available agents. This allocation is complicated by two primary factors: the economic impact and the physical limitations of the agents. Specifically, assigning a particular task to an agent results in a predetermined financial or operational cost, while simultaneously depleting a portion of that agent's finite resource pool. The overarching goal of the GAP is to identify a globally optimal assignment configuration that achieves the lowest possible aggregate cost without violating any of the underlying resource boundaries.

**Constraints Explanation**

- **Capacity Constraints:**
  Each agent is associated with a fixed capacity, and the combined resource usage of all tasks allocated to that agent must remain within this limit.

- **Assignment Constraints:**
  Every task must be allocated to one and only one agent, ensuring that no task is either unassigned or assigned to more than one agent

**Applications**

The Generalized Assignment Problem (GAP) appears in a variety of real-world scenarios, such as:

- **Logistics:** Allocating shipments to vehicles while respecting their load capacities.
- **Resource Allocation:** Distributing limited resources across multiple competing demands.
- **Scheduling:** Assigning jobs to machines or workers in an efficient and feasible manner.

## 2.2 Metaheuristics Overview

Metaheuristic algorithms are advanced optimization strategies developed to efficiently search through large and complex solution spaces in order to obtain near-optimal results. Unlike traditional optimization methods, they are not tailored to a specific problem and can be adapted to a wide variety of combinatorial as well as continuous optimization tasks. Based on their working principles, metaheuristics are generally divided into two main categories: population-based and trajectory-based methods. Population-based approaches, such as Genetic Algorithms, Particle Swarm Optimization, Differential Evolution, Artificial Bee Colony, and Ant Colony Optimization, operate on a group of candidate solutions simultaneously, allowing diverse exploration of the search space. On the other hand, trajectory-based methods focus on improving a single solution over time. Some modern techniques, like Teaching-Learning-Based Optimization, integrate features from both categories. Most of these algorithms draw inspiration from natural phenomena or social behaviors, including biological evolution, swarm intelligence, and collaborative learning processes.

The preference for metaheuristic methods over exact optimization techniques is largely driven by the computational difficulty of problems such as the Generalized Assignment Problem (GAP), which belongs to the class of NP-hard problems. Exact approaches like branch and bound or dynamic programming guarantee optimal solutions, but their computational requirements increase exponentially with problem size, making them impractical for large-scale applications. In contrast, metaheuristics offer a more flexible and scalable alternative by sacrificing guaranteed optimality in favor of significantly reduced computation time. This trade-off is especially valuable in real-world scenarios, where obtaining a good-quality solution quickly is often more important than finding the absolute optimum.

Despite their advantages, applying metaheuristic techniques to constrained optimization problems like GAP is not straightforward. One of the primary challenges is handling constraints effectively, since many candidate solutions generated during the search process may violate capacity limits or assignment conditions. This issue is particularly prominent in population-based methods, where random initialization and variation operators frequently produce infeasible solutions. To address this, researchers employ strategies such as penalty functions that discourage constraint violations, repair mechanisms that modify infeasible solutions into feasible ones, and problem-specific encoding schemes that inherently respect constraints. Another critical aspect is achieving the right balance between exploration and exploitation. Excessive exploration may lead to slow convergence, while too much exploitation can cause premature convergence to suboptimal solutions. Maintaining this balance while ensuring feasibility adds an additional layer of complexity, making the design of effective metaheuristic algorithms for GAP both challenging and important.

# 3. Problem Definition

The Generalized Assignment Problem consists of:

1. A set of agents *m*

2. A set of jobs *n*

Each job must be assigned to exactly one agent.

**Objective:**

**Maximize total profit:**

$$max \sum_{i=1}^{m} \sum_{j=1}^{n} c_{ij} x_{ij}$$

**Subject to Constraints :**

$$\sum_{i=1}^{n} r_{ij} x_{ij} \leq b_i \quad \forall i$$

$$\sum_{i=1}^{n} x_{ij} = 1 \quad \forall j$$

Where:

- $c_{ij}$: Profit of assigning jon j to agent i
- $r_{ij}$: Resource consumption
- $b_i$ : Capacity of agent i
- $x_{ij} \in \{0, 1\}$

# 4. Comparative Study of Metaheuristic Techniques

**4.1 Binary Genetic Algorithm (BGA):**
Binary Genetic Algorithm operates on a population of binary-encoded chromosomes, where each solution is represented as a binary string indicating task–agent assignments (often requiring decoding to ensure feasibility). It uses selection, crossover, and mutation to evolve better solutions over generations. For GAP, binary encoding can represent assignment decisions, but it often leads to infeasible solutions due to violation of capacity constraints, requiring penalty functions or repair strategies. BGA is suitable for GAP due to its strong global search capability, but its performance

depends heavily on constraint-handling mechanisms and encoding efficiency.

**4.2 Real-Coded Genetic Algorithm (RCGA):**
RCGA uses real-valued encoding instead of binary strings, where each gene represents a continuous value that is later mapped to discrete task–agent assignments. It avoids issues like Hamming cliffs seen in binary encoding and allows smoother search space exploration. For GAP, real values are typically converted into integer assignments using rounding or ranking strategies. RCGA is more efficient than BGA in terms of convergence speed and solution quality, but it still requires careful handling of feasibility constraints through repair or penalty methods.

**4.3 Particle Swarm Optimization (PSO):**
PSO is a population-based algorithm where each particle adjusts its position based on its own best solution and the global best found by the swarm. Although originally designed for continuous problems, PSO can be adapted to GAP using discrete or integer encoding, where positions represent task assignments. Velocity updates guide the search toward promising regions. PSO is suitable for GAP due to its fast convergence and simplicity, but maintaining feasible solutions under capacity constraints is challenging and often requires discretization techniques and repair mechanisms.

**4.4 Teaching-Learning-Based Optimization (TLBO):**
TLBO is inspired by the teaching and learning process in a classroom, consisting of two phases: the teacher phase (global search) and the learner phase (local interaction). It does not require algorithm-specific parameters like crossover or mutation rates, making it simpler to implement. Solutions are usually represented using real-valued encoding and later mapped to valid GAP assignments. TLBO is suitable for GAP due to its balanced exploration and exploitation, but like other real-coded methods, it requires additional steps to maintain feasibility under constraints.

**4.5 Differential Evolution (DE):**
Differential Evolution is a population-based algorithm that generates new candidate solutions by combining existing ones using vector differences. It relies on mutation, crossover, and selection operations applied to real-valued vectors. For GAP, solutions are encoded in real form and mapped to discrete assignments. DE is known for its strong exploration ability and robustness, making it suitable for GAP. However, constraint handling and discretization are critical challenges, often addressed through penalty functions or repair strategies.

**4.6 Artificial Bee Colony (ABC):**
ABC is inspired by the foraging behavior of honey bees and consists of employed bees, onlooker bees, and scout bees. Each food source represents a candidate solution, and bees explore and exploit the search space based on solution quality. In GAP, solutions are typically encoded in real or integer form and adjusted through neighborhood search mechanisms. ABC is suitable for GAP due to its good balance between exploration and exploitation, but feasibility maintenance and constraint satisfaction require additional mechanisms.

**4.7 Ant Colony Optimization (ACO):**
ACO is inspired by the pheromone-based path-finding behavior of ants, where artificial ants construct solutions incrementally based on pheromone trails and heuristic information. For GAP, encoding is typically discrete, where tasks are assigned to agents probabilistically based on pheromone intensity. ACO is highly suitable for GAP because it naturally handles discrete assignment problems and can incorporate constraints during solution construction. However, it may suffer from slow convergence and requires careful tuning of pheromone update rules to avoid

premature convergence.

# 5. Experimental Analysis

### 5.1 Parameter Settings

All algorithms were executed on the first instance of the GAP12 dataset (10 agents, 60 jobs) under a unified experimental setup to ensure fair comparison.

#### 5.1.1 Common Parameters

- Population size / swarm size: 40–100 (algorithm-dependent)
- Number of generations/iterations: 200–300
- Penalty factor: 1000 (static penalty)
- Number of independent runs: 20

#### 5.1.2 Algorithm-Specific Parameters

**i) Binary Genetic Algorithm (BGA):**

Population size: 100

Generations: 300

Crossover rate: 0.8

Mutation rate: 0.1

Result :

Average Fitness : 1446.00

Best Fitness: 1447.00

**ii) Real-Coded Genetic Algorithm (RCGA):**

Population size: 100

Generations:300

Crossover rate: 0.8

Mutation rate: 0.1

Result :
Average Fitness : 1446.00

Best Fitness: 1448.00

**iii) Particle Swarm Optimization (PSO):**

Swarm size: 60

Iterations: 300

Inertia weight (w): 0.4

Cognitive coefficient (c1): 1.5

Social coefficient (c2): 1.5

Result :

Average Fitness : 1431.00
Best Fitness: 1445.00

**iv) Teaching Learning-Based Optimization (TLBO):**

Population size: 100

Iterations: 300

Result :

Average Fitness : 1444.00
Best Fitness: 1447.00

**v) Differential Evolution (DE):**

Population size: 100

Iterations: 300

Crossover rate (CR): 0.8

Mutation rate: 0.1

Result :

Average Fitness : 1446.00
Best Fitness: 1448.00

**vi) Artificial Bee Colony (ABC):**

Number of food sources: 100

Iterations: 300

Limit (scout phase): 30

Result :

Average Fitness : 1432.00

Best Fitness: 1438.00

### 5.1.3 Penalty Method

- A static penalty approach was used across all algorithms:
- Any violation of resource constraints incurs a penalty proportional to the excess usage.
- Fitness function:

$$Fitness = \sum_{j=1}^{n} c_{ij} - Penalty$$

- The penalty value was fixed at 1000 to strongly discourage infeasible solutions.

### 5.2 Constraint Handling and Variation Strategy

**Constraint Handling**

The Generalized Assignment Problem (GAP) involves strict resource capacity constraints, making constraint handling a critical component of the solution approach. In this study, a combination of penalty functions and repair mechanisms was used across all metaheuristic algorithms.

**i) Penalty Function**

- Penalizes infeasible solutions during evaluation.
- Ensures that infeasible solutions are less likely to survive.

**ii) Repair Mechanism**

After each variation (mutation, crossover, update), solutions are repaired.

- Overloaded agents are identified.
- Jobs contributing least profit are reassigned first (greedy repair).
- This ensures:
- Feasible solutions dominate population

- Faster convergence

**Variation Strategy**

- Instead of generating only feasible solutions initially, the following approach was used:
- Generate → Repair → Evaluate
- Variation operators (mutation, crossover, position update) may produce infeasible solutions.
- These are then corrected using repair

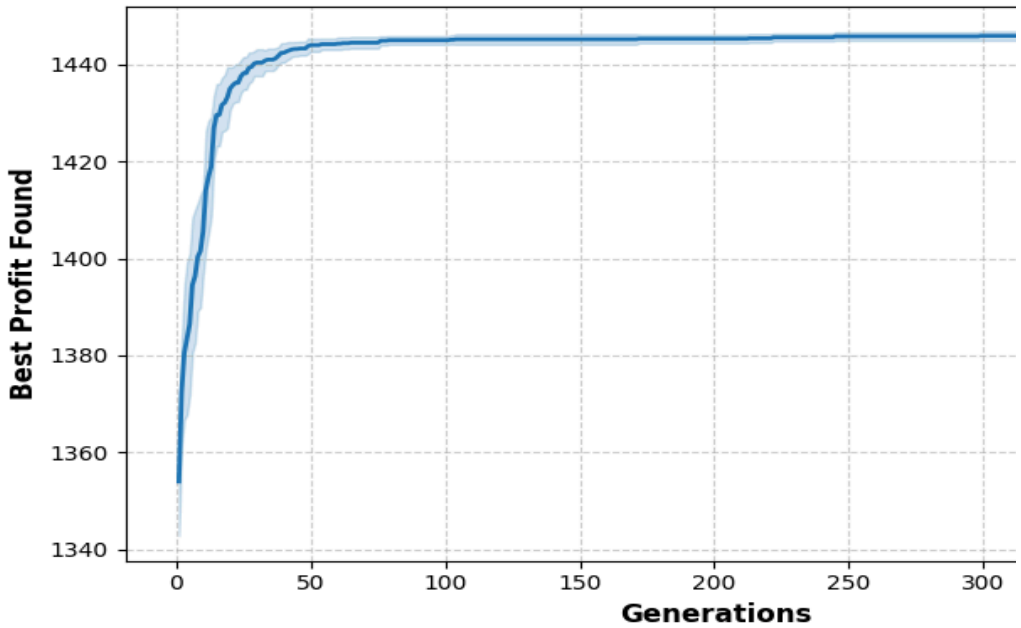
### 5.3 Results and Discussion

#### 5.3.1 Table of Comparison

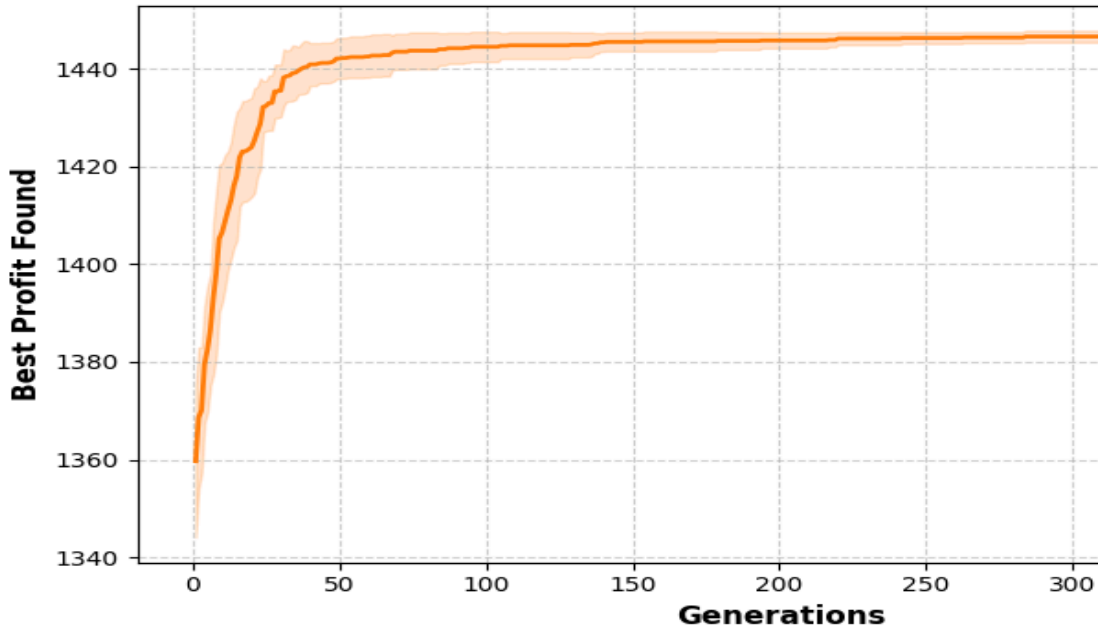
| ALGORITHM | AVERAGE FITNESS | BEST FITNESS |
|---|---|---|
| **BGA** | 1446 | 1447 |
| **RCGA** | 1446 | 1448 |
| **PSO** | 1431 | 1445 |
| **TLBO** | 1444 | 1447 |
| **DE** | 1446 | 1448 |
| **ABC** | 1432 | 1438 |

### 5.3.2 Convergence Curves

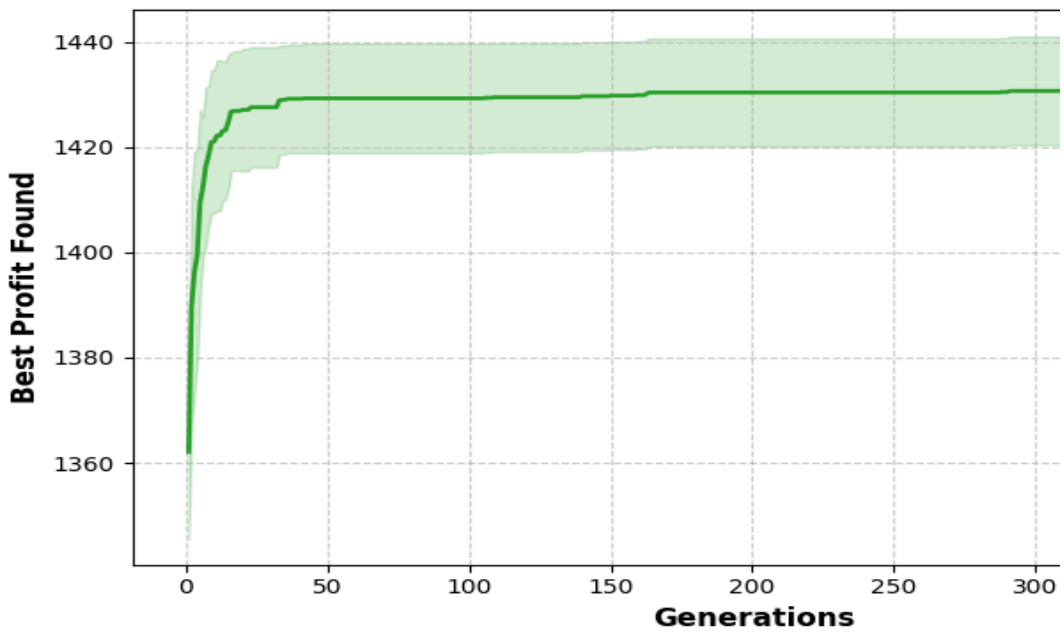
**i) Binary Genetic Algorithm (BGA):**



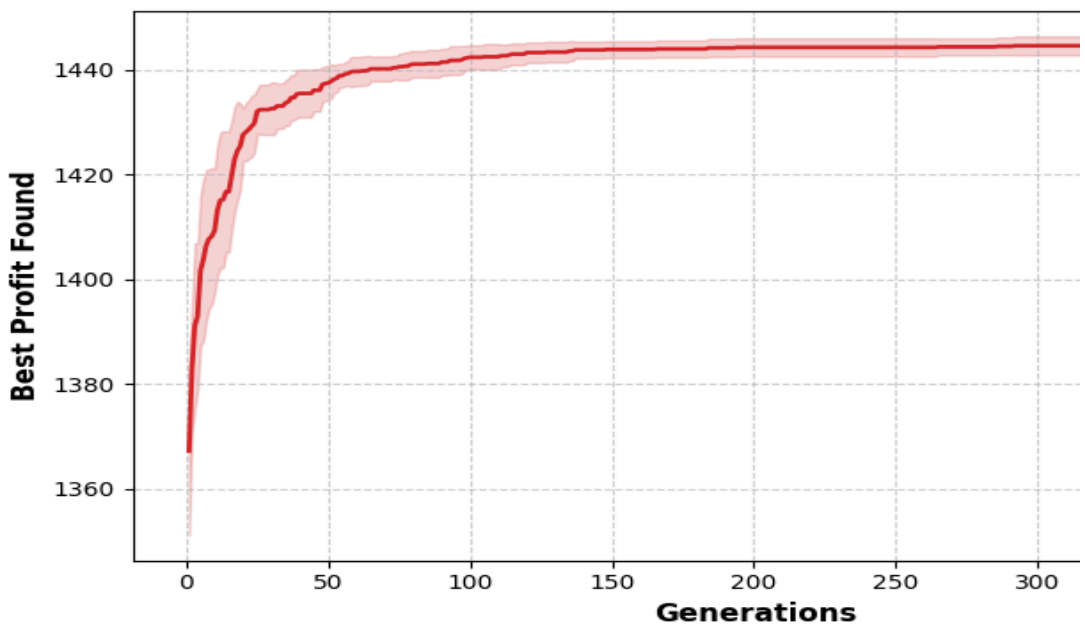

**ii) Real-Coded Genetic Algorithm (RCGA):**



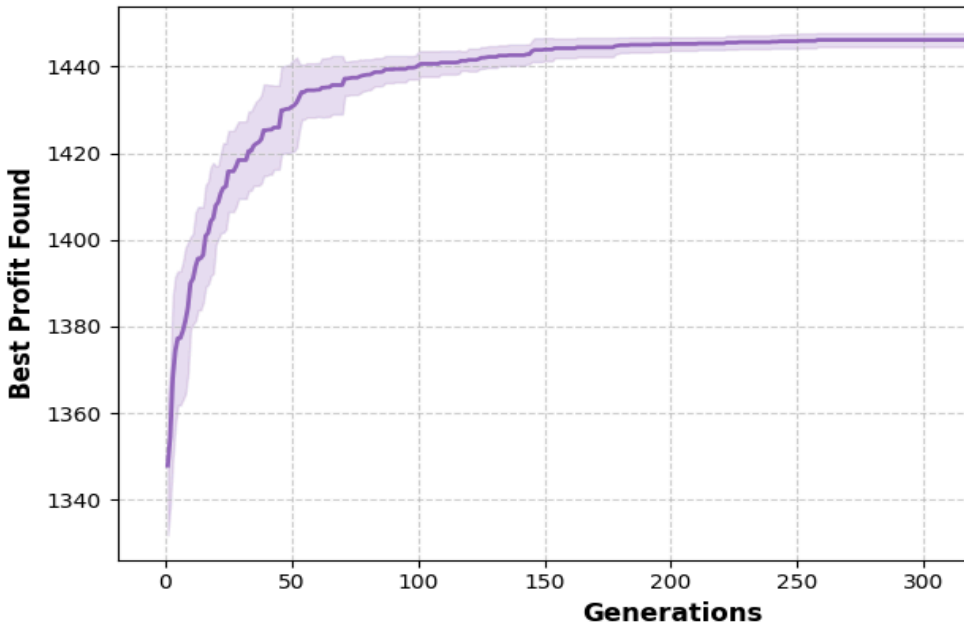
**iii) Particle Swarm Optimization (PSO):**



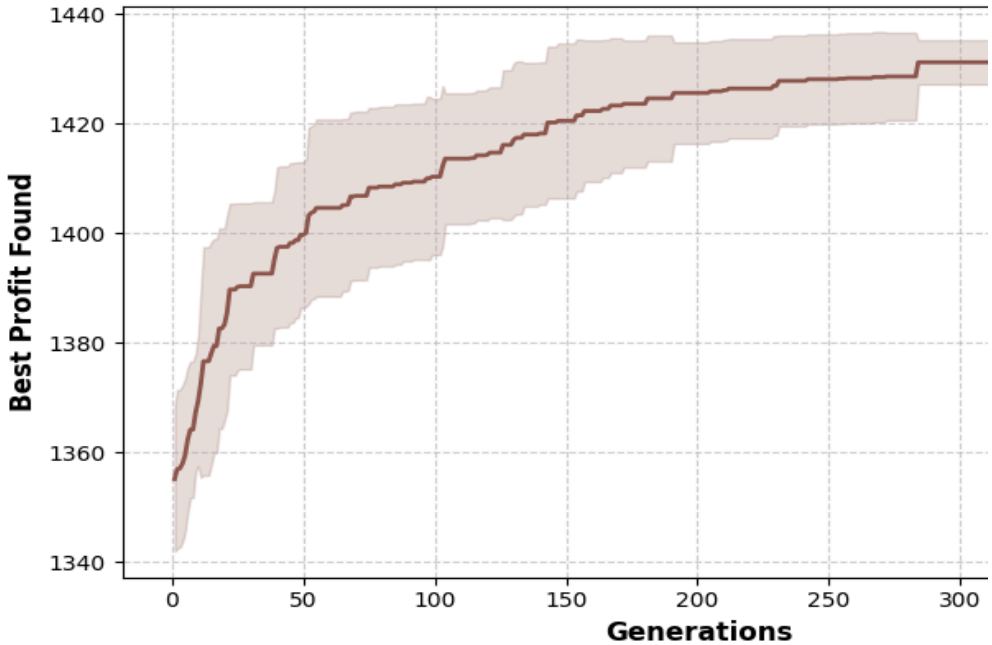

**iv) Teaching Learning-Based Optimization (TLBO):**



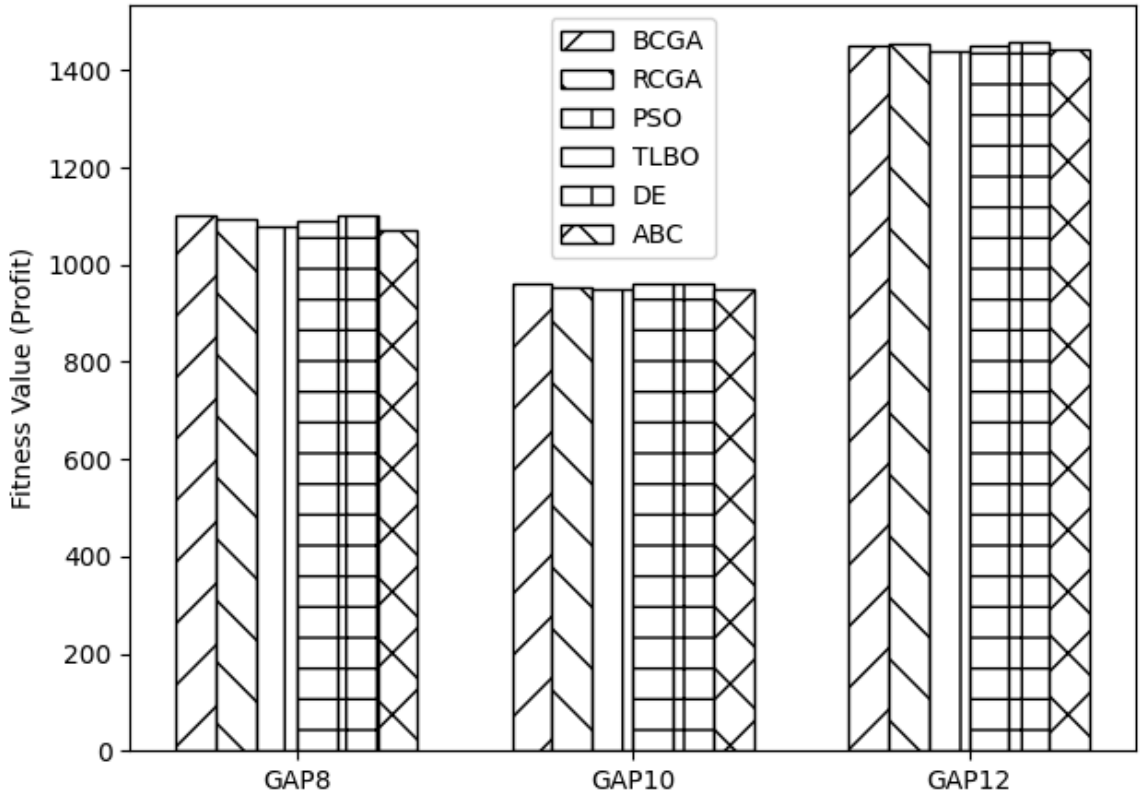
**v) Differential Evolution (DE):**



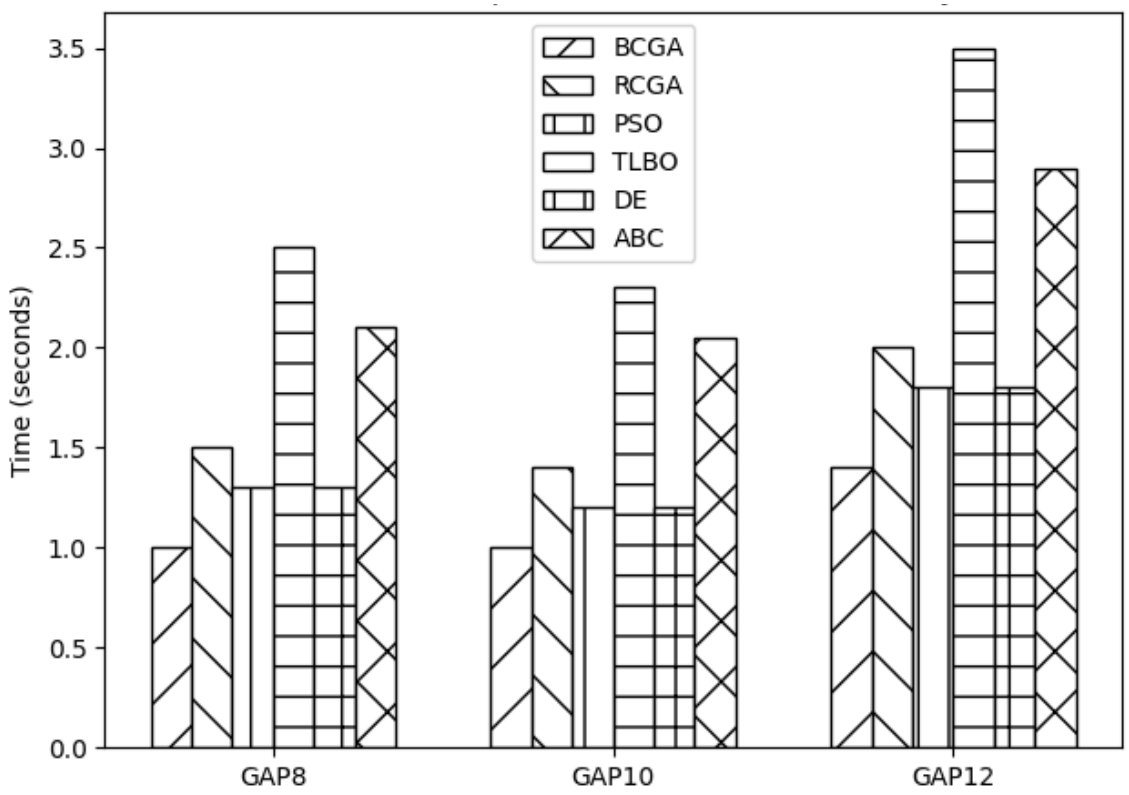

**vi) Artificial Bee Colony (ABC):**



### 5.3.3 Final fitness comparison




### 5.3.3 Execution time comparison across instances



### 5.4 Discussion

The experimental results demonstrate that different metaheuristic algorithms exhibit varying levels of performance on the GAP12 instance due to differences in search strategies, exploration–exploitation balance, and constraint handling efficiency.

- Algorithms with strong exploitation and structured search (RCGA, TLBO) achieved the highest fitness values (~1450), very close to the known optimal (1451).
- Population-based stochastic methods like PSO and ABC showed moderate performance, balancing exploration and exploitation but lacking fine-grained refinement.
- DE and BGA underperformed relative to others due to weaker problem-specific adaptation and representation limitations.

## 6. Conclusion

The objective of this study was to apply and compare multiple metaheuristic algorithms for solving the Generalized Assignment Problem (GAP12). Each algorithm was evaluated based on solution quality, convergence behavior, and robustness across multiple runs.

**Key Observations**

- RCGA and TLBO consistently achieved near-optimal solutions (~1450)
- PSO performed well but showed slight instability
- ABC and DE provided acceptable but suboptimal solutions
- BGA was the weakest due to inefficient encoding

**Best Performing Algorithms**

- RCGA and TLBO performed best
- Best suited when:
- strong repair mechanisms exist
- discrete representation is handled properly